

Programação Competitiva

Aula 2.1: Complexidade de algoritmos

Alberto Tavares Duarte Neto

01 de Setembro, 2025

Universidade de Brasília

Problema

É dado um vetor de n inteiros, com $n \leq 10^5$, e um inteiro $q \leq 10^5$. São feitas q consultas: cada consulta consiste em uma linha com dois inteiros l, r com $1 \leq l \leq r \leq n$, e você deve calcular a soma dos elementos do vetor de l até r .

Motivação

Problema

É dado um vetor de n inteiros, com $n \leq 10^5$, e um inteiro $q \leq 10^5$. São feitas q consultas: cada consulta consiste em uma linha com dois inteiros l, r com $1 \leq l \leq r \leq n$, e você deve calcular a soma dos elementos do vetor de l até r .

Solução naive

```
for(int query = 0; query < q; query++) {  
    int l, r;  
    cin >> l >> r;  
    for(int i = l; i <= r; i++) {  
        soma += v[i];  
    }  
    cout << soma << endl;  
}
```

Complexidade:

Motivação

Problema

É dado um vetor de n inteiros, com $n \leq 10^5$, e um inteiro $q \leq 10^5$. São feitas q consultas: cada consulta consiste em uma linha com dois inteiros l, r com $1 \leq l \leq r \leq n$, e você deve calcular a soma dos elementos do vetor de l até r .

Solução naive

```
for(int query = 0; query < q; query++) {  
    int l, r;  
    cin >> l >> r;  
    for(int i = l; i <= r; i++) {  
        soma += v[i];  
    }  
    cout << soma << endl;  
}
```

Complexidade: $O(nq)$.

Soma de prefixos

Calculamos um vetor p da seguinte forma:

$$p[i] = \begin{cases} v[0], & \text{se } i = 0, \\ v[i] + p[i - 1], & \text{caso contrário.} \end{cases} \quad (0.1)$$

Soma de prefixos

Calculamos um vetor p da seguinte forma:

$$p[i] = \begin{cases} v[0], & \text{se } i = 0, \\ v[i] + p[i - 1], & \text{caso contrário.} \end{cases} \quad (0.1)$$

O que é o valor $p[r] - p[l - 1]$. É exatamente

$$\begin{aligned} &v[0] + v[1] + \cdots + v[l - 1] + v[l] + \cdots + v[r] - \\ &(v[0] + v[1] + \cdots + v[l - 1]) = \\ &v[l] + \cdots + v[r] . \end{aligned}$$

Soma de prefixos

Calculamos um vetor p da seguinte forma:

$$p[i] = \begin{cases} v[0], & \text{se } i = 0, \\ v[i] + p[i - 1], & \text{caso contrário.} \end{cases} \quad (0.1)$$

O que é o valor $p[r] - p[l - 1]$. É exatamente

$$\begin{aligned} &v[0] + v[1] + \cdots + v[l - 1] + v[l] + \cdots + v[r] - \\ &(v[0] + v[1] + \cdots + v[l - 1]) = \\ &v[l] + \cdots + v[r] . \end{aligned}$$

Soma de prefixos

Calculamos um vetor p da seguinte forma:

$$p[i] = \begin{cases} v[0], & \text{se } i = 0, \\ v[i] + p[i - 1], & \text{caso contrário.} \end{cases} \quad (0.1)$$

O que é o valor $p[r] - p[l - 1]$. É exatamente

$$\begin{aligned} &v[0] + v[1] + \cdots + v[l - 1] + v[l] + \cdots + v[r] - \\ &(v[0] + v[1] + \cdots + v[l - 1]) = \\ &v[l] + \cdots + v[r] . \end{aligned}$$

Soma de prefixos

Calculamos um vetor p da seguinte forma:

$$p[i] = \begin{cases} v[0], & \text{se } i = 0, \\ v[i] + p[i - 1], & \text{caso contrário.} \end{cases} \quad (0.1)$$

Solução linear

```
psum[0] = 0;
for(int i = 1; i < n; i++) psum[i] = v[i] + psum[i-1];

for(int i = 0; i < q; i++) {
    int res = psum[r];
    if(l-1 >= 0) res -= psum[l-1];
    cout << res << endl;
}
```

Complexidade: $O(n + q)$.

Outras operações

A fórmula $p[r] + (-p[l - 1])$ assume as seguintes propriedades:

Outras operações

A fórmula $p[r] + (-p[l - 1])$ assume as seguintes propriedades:

- (i) Associatividade da operação.

Outras operações

A fórmula $p[r] + (-p[l - 1])$ assume as seguintes propriedades:

- (i) Associatividade da operação.
- (ii) Existência da identidade (neste caso, o 0).

Outras operações

A fórmula $p[r] + (-p[l - 1])$ assume as seguintes propriedades:

- (i) Associatividade da operação.
- (ii) Existência da identidade (neste caso, o 0).
- (iii) Para todo elemento x , existe um elemento $-x$ tal que $x + (-x) = 0$.

Outras operações

A fórmula $p[r] + (-p[l - 1])$ assume as seguintes propriedades:

- (i) Associatividade da operação.
- (ii) Existência da identidade (neste caso, o 0).
- (iii) Para todo elemento x , existe um elemento $-x$ tal que $x + (-x) = 0$.

Outras operações com essas propriedades: multiplicação (números, matrizes), ou exclusivo (XOR).

Outras operações

A fórmula $p[r] + (-p[l - 1])$ assume as seguintes propriedades:

- (i) Associatividade da operação.
- (ii) Existência da identidade (neste caso, o 0).
- (iii) Para todo elemento x , existe um elemento $-x$ tal que $x + (-x) = 0$.

Outras operações com essas propriedades: multiplicação (números, matrizes), ou exclusivo (XOR).

Soma de prefixos com XOR

```
psum[0] = v[0];  
for(int i = 1; i < n; i++) psum[i] = v[i] ^ psum[i-1];  
  
for(int i = 0; i < q; i++) {  
    int res = psum[r];  
    if(l-1 >= 0) res ^= psum[l-1];  
    cout << res << endl;  
}
```

Problema

É dado um vetor de n inteiros e um inteiro x . Quantos subvetores (contíguos) tem soma exatamente x ?

Motivação

Problema

É dado um vetor de n inteiros e um inteiro x . Quantos subvetores (contíguos) tem soma exatamente x ?

A solução *naive* neste caso consiste em iterar todos os subvetores. A quantidade de subvetores é exatamente $n(n-1)/2$.

Solução naive

```
for(int l = 0; l < n; l++) {  
    for(int r = l; r < n; r++) {  
        soma += v[r];  
        if(soma == x) resposta++;  
    }  
}
```

Complexidade:

Motivação

Problema

É dado um vetor de n inteiros e um inteiro x . Quantos subvetores (contíguos) tem soma exatamente x ?

A solução *naive* neste caso consiste em iterar todos os subvetores. A quantidade de subvetores é exatamente $n(n-1)/2$.

Solução naive

```
for(int l = 0; l < n; l++) {  
    for(int r = l; r < n; r++) {  
        soma += v[r];  
        if(soma == x) resposta++;  
    }  
}
```

Complexidade: $O(n^2)$.

Dois ponteiros

Propriedade importante do problema: Fixe o ponteiro l . Ao aumentar o r , a soma só aumenta. Ao aumentar l , a soma só diminui.

Dois ponteiros

Propriedade importante do problema: Fixe o ponteiro l . Ao aumentar o r , a soma só aumenta. Ao aumentar l , a soma só diminui.

Em outras palavras:

- Se l está fixo, a soma é monotônica como função de r .
- Se r está fixo, a soma é monotônica como função de l .

Dois ponteiros

Propriedade importante do problema: Fixe o ponteiro l . Ao aumentar o r , a soma só aumenta. Ao aumentar l , a soma só diminui.

Em outras palavras:

- Se l está fixo, a soma é monotônica como função de r .
- Se r está fixo, a soma é monotônica como função de l .

Seja S a soma de l até r . Analisemos os seguintes casos:

Dois ponteiros

Propriedade importante do problema: Fixe o ponteiro l . Ao aumentar o r , a soma só aumenta. Ao aumentar l , a soma só diminui.

Em outras palavras:

- Se l está fixo, a soma é monotônica como função de r .
- Se r está fixo, a soma é monotônica como função de l .

Seja S a soma de l até r . Analisemos os seguintes casos:

- Se $S > x$, nenhum intervalo $[l, r']$ com $r' > r$ vai ter soma igual a x . Devemos aumentar l .

Dois ponteiros

Propriedade importante do problema: Fixe o ponteiro l . Ao aumentar o r , a soma só aumenta. Ao aumentar l , a soma só diminui.

Em outras palavras:

- Se l está fixo, a soma é monotônica como função de r .
- Se r está fixo, a soma é monotônica como função de l .

Seja S a soma de l até r . Analisemos os seguintes casos:

- Se $S > x$, nenhum intervalo $[l, r']$ com $r' > r$ vai ter soma igual a x . Devemos aumentar l .
- Se $S \leq x$, nenhum intervalo $[l', r]$ com $l' > l$ vai ter soma igual a x . Devemos aumentar l . No caso de igualdade, aumentamos a resposta em 1.

Dois ponteiros

Propriedade importante do problema: Fixe o ponteiro l . Ao aumentar o r , a soma só aumenta. Ao aumentar l , a soma só diminui.

Em outras palavras:

- Se l está fixo, a soma é monotônica como função de r .
- Se r está fixo, a soma é monotônica como função de l .

Seja S a soma de l até r . Analisemos os seguintes casos:

- Se $S > x$, nenhum intervalo $[l, r']$ com $r' > r$ vai ter soma igual a x . Devemos aumentar l .
- Se $S \leq x$, nenhum intervalo $[l', r]$ com $l' > l$ vai ter soma igual a x . Devemos aumentar l . No caso de igualdade, aumentamos a resposta em 1.

Dois ponteiros

Solução com dois ponteiros

```
int r = -1, l = 0, soma = 0, resposta = 0;
while(r < n) { // cuidado com x = 0
    if(soma >= x) {
        if(soma == x) resposta++;
        soma -= v[l]; l++;
    }
    else if(soma < x) {
        r++;
        if(r < n) soma += v[r];
    }
}
```

Dois ponteiros

Solução com dois ponteiros

```
int r = -1, l = 0, soma = 0, resposta = 0;
while(r < n) { // cuidado com x = 0
    if(soma >= x) {
        if(soma == x) resposta++;
        soma -= v[l]; l++;
    }
    else if(soma < x) {
        r++;
        if(r < n) soma += v[r];
    }
}
```

Em geral, qualquer valor que seja monotônico em função de l e r pode ser calculado com esta técnica.

Dois ponteiros

Solução com dois ponteiros

```
int r = -1, l = 0, soma = 0, resposta = 0;
while(r < n) { // cuidado com x = 0
    if(soma >= x) {
        if(soma == x) resposta++;
        soma -= v[l]; l++;
    }
    else if(soma < x) {
        r++;
        if(r < n) soma += v[r];
    }
}
```

Em geral, qualquer valor que seja monotônico em função de l e r pode ser calculado com esta técnica.

Complexidade: $O(n)$.

Podemos escrever esse código de uma maneira alternativa:

Dois ponteiros

Podemos escrever esse código de uma maneira alternativa:

Código alternativo para dois ponteiros

```
int r = -1, l = 0, soma = 0, resposta = 0;
while(r+1 < n) {
    r++;
    soma += v[r];
    while(soma >= x) {
        if(soma == x) resposta++;
        soma -= v[l]; l++;
    }
}
```